# multiCooperativa

## App móvil multi-cooperativa con asistente de IA

Trabajo Fin de Máster · Máster de Desarrollo con IA (BIG school)

Juan Chaparro García · Octubre 2026

# El problema: un socio poco informado

### Convocatorias en papel

Cartas y tablón de anuncios: el socio se entera tarde o no se entera.

### Documentos fuera de alcance

Estatutos, reglamentos y actas no están a mano.

### Descuadres detectados tarde

Albaranes y facturas en papel: impagos y cobros pendientes se ven tarde o nunca.

**Objetivo: conectar a cada cooperativa con sus socios y controlar albaranes y facturas con un asistente de IA que respeta los permisos**

# La solución y sus cuatro perfiles

Una app estándar para cualquier cooperativa; un usuario, varias cooperativas, datos aislados.

***El backend decide; la app solo oculta.***

### Socio

Informarse, votar, leer documentos y ver su capital

### Consejo rector

Comunicados, asambleas, vigilar pagos y cobros

### Gestor

Registrar y conciliar albaranes y facturas

### Administrador

Dar acceso con invitaciones, asignar perfiles y marca

# Módulos del MVP: hecho y pendiente

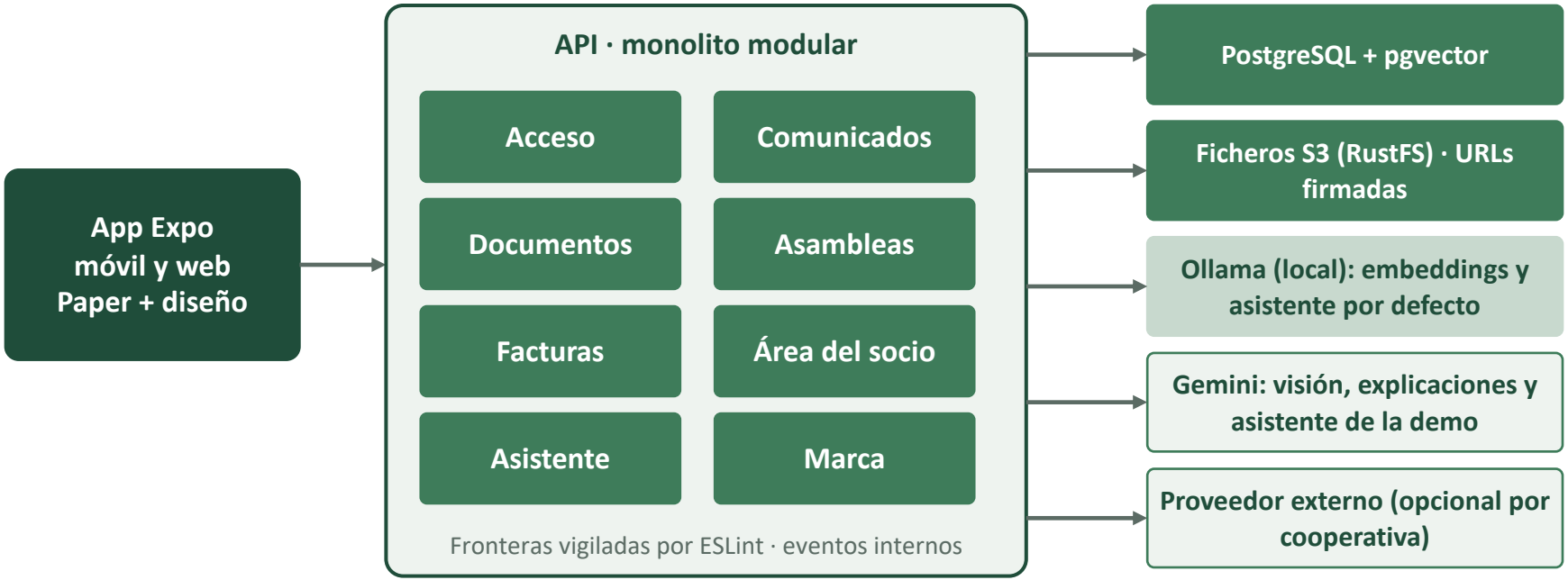
## Implementado: los 7 módulos

- Acceso y perfiles, miembros e invitaciones por enlace
- Comunicados
- Documentos y búsqueda semántica
- Asambleas y votación
- Facturas y albaranes con IA; proveedores y clientes
- Área del socio (capital y aportaciones)
- Asistente de IA con citas, historial, facturación agregada y socios; modelo local o externo
- Marca por cooperativa (color) y sistema de diseño

## Trabajo futuro

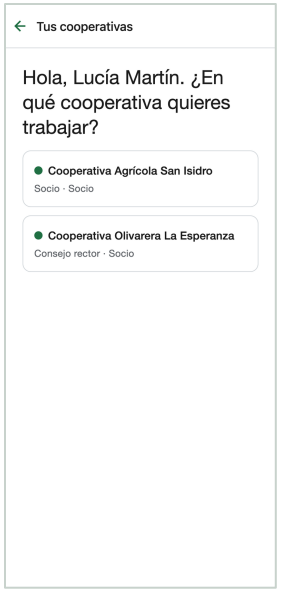
- Notificaciones push
- Sentry
- Tests de componentes (RNTL) y E2E en CI
- Reclamaciones de facturas
- Binarios nativos (EAS Build)
- Invitaciones por correo

# Arquitectura: monolito modular

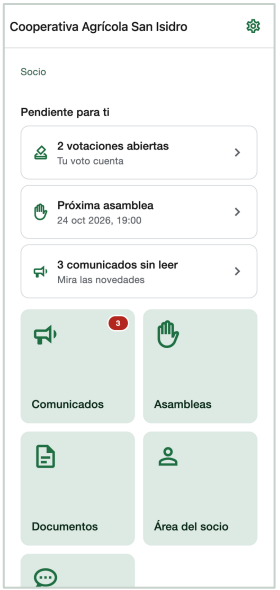



Todo en Docker en un servidor propio · acceso público solo a la API, por Cloudflare Tunnel

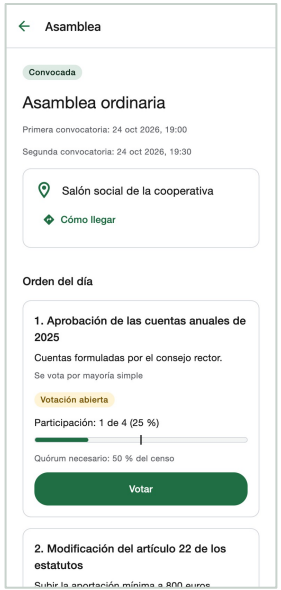
# Interfaz: del brief al sistema de diseño




Selector: color de cada cooperativa




Inicio: "Pendiente para ti"




Asamblea: mapa y quórum




Facturas y albaranes

## Del brief a la app

- Brief de interfaz (docs/05)
- Herramienta de diseño con IA: tokens, 8 colores, unos 40 componentes
- Análisis de brechas (docs/06)
- Orquestador y 7 subagentes en paralelo, uno por módulo
- Kit en src/ui; las pantallas no usan Paper directamente

# Decisiones clave: 26 ADR

| | |
|---|---|
| 001 | App híbrida con Expo |
| 002 | Monolito modular + Clean Architecture |
| 003 | Multi-cooperativa, BD compartida |
| 004 | Permisos por perfil, módulo y alcance |
| 005 | Ficheros S3 tras un puerto |
| 006 | Embeddings locales (bge-m3, Ollama) |
| 007 | Claude para visión (sustituido por 009) |
| 008 | Código en inglés, UI localizada |
| 009 | Gemini para visión y lenguaje |
| 010 | Embeddings Gemini (alternativa en la nube) |
| 011 | README raíz bilingüe |
| 012 | Servidor propio + Cloudflare Tunnel |
| 013 | Modelo local para el asistente |
| 014 | Modelo del asistente por cooperativa |
| 015 | qwen3:4b-instruct por defecto |
| 016 | Invitaciones por enlace del administrador |
| 017 | Dominio propio para la web |
| 018 | Color por cooperativa (logo: 021) |
| 019 | Proveedor externo por cooperativa |
| 020 | Interfaz con react-native-paper (MD3) |
| 021 | Sin logo por cooperativa (el color se mantiene) |
| 022 | Búsqueda híbrida y umbral de similitud |
| 023 | Calidad RAG: benchmark y recuperación |
| 024 | Sistema de diseño de la interfaz |
| 025 | NIF del socio y cruce socio–tercero |
| 026 | Gemini para el asistente de la demo |

# Seguridad y multi-cooperativa

### Denegar por defecto

Cada caso de uso comprueba el permiso; la app solo oculta

### Aislamiento

404 si el recurso es de otra cooperativa, 403 sin permiso; también en RAG y herramientas

### Contraseñas

Argon2id, bloqueo por intentos fallidos y JWT con refresh rotatorio

### Ficheros y validación

URLs firmadas que caducan; el servidor valida cada entrada, también el contraste AA del color

### CORS y secretos

Orígenes permitidos; secretos fuera del repo; claves de proveedor cifradas (AES-GCM)

### Tests de seguridad

Matriz perfiles x módulos y aislamiento entre cooperativas

# IA en el producto

## Asistente: local o en la nube

Ollama en el servidor propio (qwen3:4b-instruct) por defecto; Gemini Flash-Lite en la demo publicada (ADR-026); modelo elegible por cooperativa o proveedor externo

## RAG y herramientas con citas

1 búsqueda en documentos (RAG) y 17 consultas a datos, de solo lectura y con los permisos del usuario: facturación agregada en BD, asambleas, comunicados, socios y cruce socio–tercero por NIF; cifras citadas

## Gemini para visión

Extrae facturas (esquema Zod y confirmación humana) y explica discrepancias; con reintentos y alternativa

## Conversación

Historial que se reabre y se elimina; respuestas en Markdown; "¿cuánto he aportado?" y proveedores/clientes

Embeddings locales con bge-m3 (Ollama) · Gemini para visión, explicaciones y el asistente de la demo

# IA en el desarrollo

**248 / 249**

commits co-autorados con Claude (99 %)

63 historias de usuario
26 ADR

- Spec Driven Development: especificación antes del código
- CLAUDE.md como contrato con el agente; ESLint protege las fronteras
- TDD en el dominio; IDs de requisitos en tests y commits
- Agentes en paralelo, cada uno en su git worktree
- Skills de la comunidad fijadas en el repo, con reglas de precedencia
- Diseño con IA: brief, herramienta de diseño, análisis de brechas y migración con orquestador y 7 subagentes
- Evaluaciones con el modelo real (60 preguntas)
- Benchmark de modelos locales antes de decidir

# Calidad

**1504**

tests unitarios en 136 ficheros

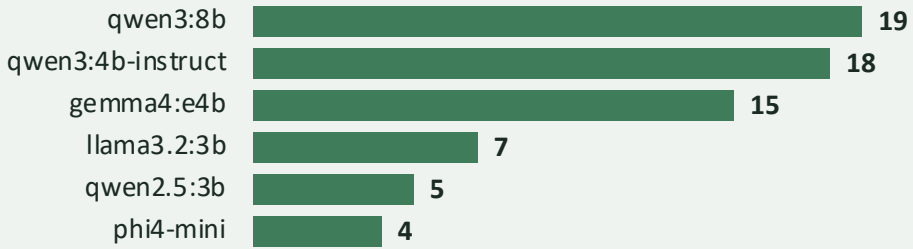
**611**

tests de integración en 32 ficheros

## Puerta de calidad y CI

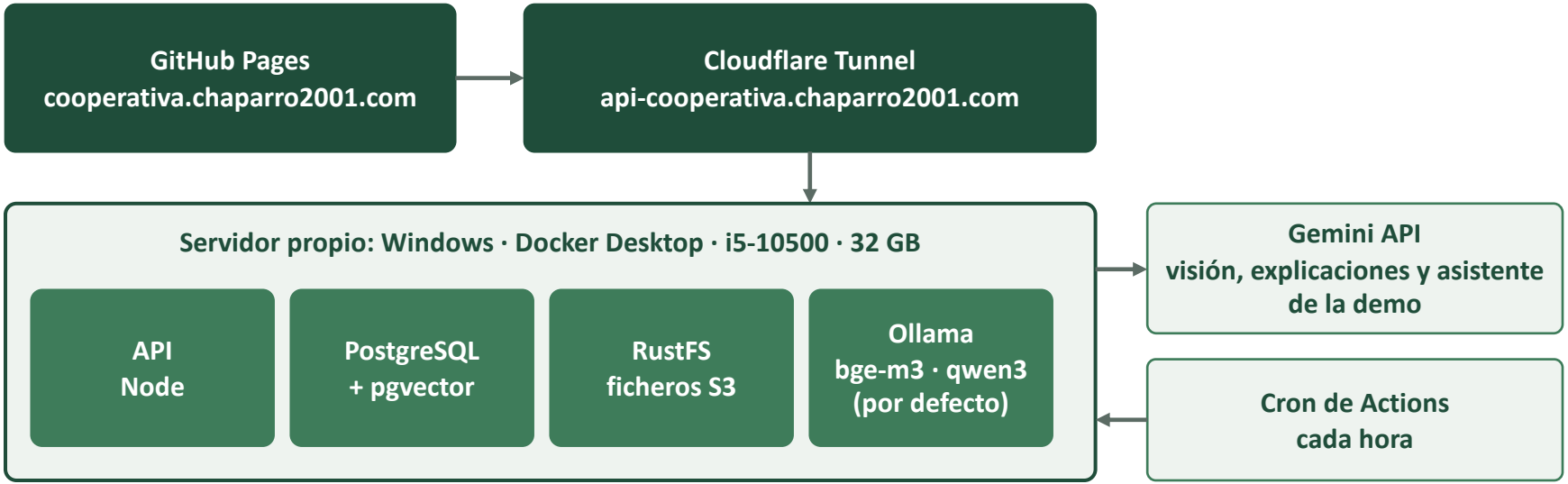
Husky en pre-commit; Actions: formato, lint, tipos, tests

## Benchmark local del asistente: aciertos de 22




Benchmark de 22 preguntas con el que se eligió el modelo local · Mac de 12 CPU
Conjunto actual: 60 preguntas · qwen3:4b-instruct 47/60 · Gemini Flash-Lite 53/60

# Despliegue gratuito, sin tarjeta




**Coste: 0 EUR** · Render y Railway pedían tarjeta; Neon, Supabase y Render siguen documentados

Límites: solo responde con el servidor encendido · sin copias de seguridad · no subir datos reales

# Demo

**URL:** https://cooperativa.chaparro2001.com

Contraseña de la demo: <contraseña demo>

| Perfil | Usuario | Qué se enseña |
|---|---|---|
| **Socio** | socio@sanisidro.test | "Pendiente para ti", comunicados, documentos, asamblea (Cómo llegar, quórum) y voto, área del socio, asistente |
| **Gestor** | gestor@sanisidro.test | Facturas, escaneo con IA, discrepancias, proveedores y clientes; asistente: "mayores proveedores del año" |
| **Consejo** | consejo@sanisidro.test | Comunicados, asambleas, vigilar pagos y cobros; asistente: capital y censo de socios |
| **Administrador** | admin@sanisidro.test | Miembros e invitación, ajustes (apariencia, facturación, asistente IA), historial |

# Lecciones aprendidas y trabajo futuro

## Lecciones

- Desde casi 15 años de iOS nativo: aprendizaje de todo el stack, de Docker al backend, la base de datos y la IA
- Tras el Máster oficial en IA de UNIR, este máster amplió ese conocimiento al desarrollo con IA
- La cuota de Gemini llevó al modelo local; en el servidor, la CPU lo hizo inviable (>30 s) y la demo volvió a Gemini
- Latencia en CPU medida: el 4b es más rápido; el 8b, algo más preciso
- Agentes en paralelo exigen disciplina de integración
- Evaluar con el modelo real, no solo offline (32/60 → 47/60)
- Especificar primero, CLAUDE.md y ESLint protegen la arquitectura
- Diseño generado con IA: análisis de brechas antes de migrar

## Trabajo futuro

- Notificaciones push
- Sentry
- Tests de componentes (RNTL) y E2E en CI
- Reclamaciones de facturas
- Binarios nativos (EAS Build)
- Invitaciones por correo

# Gracias

Repositorio: github.com/chaparro2001/multiCooperativa

Demo: cooperativa.chaparro2001.com

API: api-cooperativa.chaparro2001.com

Vídeo: <URL vídeo>

Juan Chaparro García · TFM Máster de Desarrollo con IA · BIG school · Octubre 2026